Untitled-1.cpp

```cpp
#include <iostream>
#include <vector>
#include <cuda_runtime.h>
// 1. Warp 级别求最大值
__inline__ __device__ int warpReduceMax(int val) {
    for (int offset = warpSize / 2; offset > 0; offset /= 2) {
        val = max(val, __shfl_down_sync(0xffffffff, val, offset));
    }
    return val;
}
// 2. Block 级别求最大值
__inline__ __device__ int blockReduceMax(int val) {
    static __shared__ int shared[32];
    int lane = threadIdx.x % warpSize, wid = threadIdx.x / warpSize;
    val = warpReduceMax(val); // Warp 内规约
    if (lane == 0) shared[wid] = val;
    __syncthreads();
    // 取出各 Warp 的最大值，再次进行 Warp 规约
    val = (threadIdx.x < blockDim.x / warpSize) ? shared[lane] : INT_MIN;
    if (wid == 0) val = warpReduceMax(val);
    return val;
}
// 3. 全局 Kernel
__global__ void reduceMaxKernel(int* d_in, int* d_out, int N) {
    int max_val = INT_MIN; // 求最大值，初始值必须是极小值
    for (int i = blockIdx.x * blockDim.x + threadIdx.x; i < N; i += blockDim.x * gridDim.x) {
        max_val = max(max_val, d_in[i]);
    }
    max_val = blockReduceMax(max_val);
    if (threadIdx.x == 0) atomicMax(d_out, max_val); // Block 间原子比较
}
int main() {
    int N = 1000000;
    std::vector<int> h_in(N, 1);
    h_in[50024] = 9999;
    int *d_in, *d_out, h_out = 0;
    cudaMalloc(&d_in, N * sizeof(int));
    cudaMalloc(&d_out, sizeof(int));
    cudaMemcpy(d_in, h_in.data(), N * sizeof(int), cudaMemcpyHostToDevice);
    // ⚠️ 关键点：求最大值的全局输出必须初始化为极小值
    int init_val = INT_MIN;
    cudaMemcpy(d_out, &init_val, sizeof(int), cudaMemcpyHostToDevice);
    // 启动 Kernel (最大 1024 个 Block 配合跨步循环足够处理千万级数据)
    reduceMaxKernel<<<256, 256>>>(d_in, d_out, N);
    cudaMemcpy(&h_out, d_out, sizeof(int), cudaMemcpyDeviceToHost);
    std::cout << "Max Value Found: " << h_out << std::endl;
    cudaFree(d_in); cudaFree(d_out);
    return 0;
}
```